Ejercicios de Relaciones entre clases

Programación — DAW

Ricardo Pérez López
IES Doñana

Curso 2025/2026

1. Dibujar un diagrama de clases UML que represente el modelo estático de cada uno de los sistemas del boletín de ejercicios de *Programación orientada a objetos*, incluyendo las clases, sus atributos, sus métodos y las relaciones entre las clases. Indicar de qué tipo son las relaciones (dependencia, agregación o composición) que se establecen entre las clases.
2. Escribir una función que reciba una lista de figuras y que calcule la suma de las áreas de cada figura sin que necesite saber de qué tipo es cada una (aplicar polimorfismo).
3. Crear la clase abstracta `Animal` con el método abstracto `emitir_sonido`, y dos subclases suyas `Perro` y `Gato` que implementen ese método abstracto para permitir que las clases sean instanciables.
4. Crear la clase abstracta `Volador` con el método abstracto `volar()`, y dos subclases suyas `Pajaro` y `Avion` que implementen ese método abstracto para permitir que las clases sean instanciables.
5. Crear la clase `Vehiculo` cuyo constructor reciba la marca y el modelo, y una subclase suya `Coche` cuyo constructor reciba, además de los dos anteriores, el número de puertas. Usar `super()` en el constructor de la subclase para inicializar adecuadamente los atributos de `Vehiculo`.
6. Diseñar la clase `Hora`, que representa un instante de tiempo compuesto por la hora (de 0 a 23) y los minutos. Dispone de los métodos:
   - `__init__(`*hora*`,`*minutos*`)`: construye un objeto con los datos pasados como argumentos.
   - `inc()`: incrementa el instante en un minuto y no devuelve nada.

- `set_minutos(valor)`: asigna un valor (si es válido) a los minutos. Devuelve `True` o `False` según se haya podido modificar los minutos o no.
- `set_hora(valor)`: asigna un valor (si está comprendido entre 0 y 23) a la hora. Devuelve `True` o `False` según se haya podido cambiar la hora o no.
- `__str__()`: devuelve una cadena con la representación de la hora.

7. A partir de la clase `Hora` diseñada en el ejercicio anterior, implementar la clase `HoraExacta`, que incluye en la hora los segundos. Además de los métodos heredados desde la clase `Hora`, dispondrá de:

   - `__init__(hora, minutos, segundos)`: construye un objeto con los datos pasados como argumentos.
   - `set_segundos(valor)`: asigna un valor (si está comprendido entre 0 y 59) a los segundos. Devuelve `True` o `False` según se haya podido cambiar los segundos o no.
   - `inc()`: incrementa la hora en un segundo.

8. Añadir a la clase `HoraExacta` un método que compare si dos horas (la invocante y otra pasada como argumento al método) son iguales o distintas. ¿Cómo debería llamarse ese método?

9. Crear la clase abstracta `Instrumento` que almacena en una lista las notas musicales de una melodía (dentro de una misma octava). El método concreto `add` añade nuevas notas musicales. La clase también dispone del método abstracto `interpretar` con la siguiente signatura:

   ```
   interpretar() -> None
   ```

   que, en cada subclase que herede de `Instrumento`, mostrará por la salida las notas musicales según las interprete. Las notas serán constantes estáticas definidas dentro de la clase `Nota`, de la siguiente forma:

   ```
   class Nota:
       DO = 'do'
       RE = 're'
       MI = 'mi'
       FA = 'fa'
       SOL = 'sol'
       LA = 'la'
       SI = 'si'
   ```

10. Crear la clase `Piano` como subclase de la clase `Instrumento` del ejercicio anterior.

11. Diseñar un **sistema de gestión de biblioteca** con las siguientes clases:

- Libro
- Usuario
- Prestamo

Requisitos:

- Un usuario puede tener varios préstamos (asociación).
- Un préstamo incluye un libro (composición o agregación, a decidir).

Métodos:

- Prestar libro.
- Devolver libro.
- Mostrar libros prestados.

Añadir:

- Encapsulación.
- Generalización (por ejemplo, `UsuarioPremium`).
- Polimorfismo.

12. Diseñar un videojuego de lucha con distintos tipos de personajes. El sistema estará formado por:

    - Una clase abstracta `Personaje` con un método abstracto `atacar()`.
    - Subclases `Guerrero`, `Mago` y `Arquero`.

    Añadir:

    - Composición: inventario de armas.
    - Polimorfismo en combate.
    - Encapsulación de vida.